

```
%macro write 2 ; Macro for writing output to stdout
    mov rax, 1 ; syscall number for sys_write
    mov rdi, 1 ; file descriptor for stdout
    mov rsi, %1 ; address of string to write
    mov rdx, %2 ; length of string
    syscall ; invoke system call
%endmacro
```

```
%macro read 2 ; Macro for reading input from stdin
    mov rax, 0 ; syscall number for sys_read
    mov rdi, 0 ; file descriptor for stdin
    mov rsi, %1 ; address of buffer to store input
    mov rdx, %2 ; size of buffer
    syscall ; invoke system call
%endmacro
```

```
section .data
    question db "Write a switch case driven X86/64 ALP to perform 64-bit hexadecimal arithmetic
operations (+, -, *, /) using suitable macros. Define procedure for each operation.", 10,
'Name:-Sumaanyu', 10, 'roll:-7256 ', 10, 'Date Of Performance:-10/02/2025', 10, 10 ; Instruction and
metadata
```

```
    questionlen equ $-question ; Length of the question string
    msg db "-----MENU-----", 10
    db "\t1. Addition ", 10
    db "\t2. Subtraction ", 10
    db "\t3. Division ", 10
    db "\t4. Multiplication ", 10
    db "\t5. Exit ", 10
    db "Enter your choice : "
    msglen equ $-msg ; Length of the menu string
    msg1 db "Number : " ; Prompt for entering numbers
    len equ $-msg1 ; Length of prompt
    endl db 10 ; Newline character
```

```
section .bss
    choice resb 2 ; Buffer to store user choice
    num resb 17 ; Buffer to store input number

    num1 resq 1 ; Storage for first number
    num2 resq 1 ; Storage for second number
    temp resb 16 ; Temporary buffer
```

```
section .text
    global _start
_start:
    write question, questionlen ; Display question prompt
main_menu:
    write msg, msglen ; Display menu
    read choice, 2 ; Read user choice

    cmp byte[choice], "1" ; Compare input with '1'
    je case1 ; Jump to addition if match
```

```

cmp byte[choice], "2"
je case2 ; Jump to subtraction if match

cmp byte[choice], "3"
je case3 ; Jump to division if match

cmp byte[choice], "4"
je case4 ; Jump to multiplication if match

cmp byte[choice], "5"
je case5 ; Exit program if match
jmp main_menu ; Loop back to menu

```

```

case1:
    call addition ; Perform addition
    jmp main_menu

case2:
    call subtraction ; Perform subtraction
    jmp main_menu

case3:
    call division ; Perform division
    jmp main_menu

case4:
    call multiplication ; Perform multiplication
    jmp main_menu

case5:
    mov rax, 60 ; syscall number for exit
    mov rdi, 0 ; Exit status
    syscall ; Invoke system call

```

```

addition:
    write msg1, len ; Prompt for first number
    read num, 17 ; Read first number

    call ascii_hex ; Convert ASCII to hex
    mov [num1], rbx ; Store number

    write msg1, len ; Prompt for second number
    read num, 17 ; Read second number

    call ascii_hex ; Convert ASCII to hex
    mov [num2], rbx ; Store number

    mov rax, [num1]
    mov rbx, [num2]

    add rbx, rax ; Perform addition

    call display ; Display result
    ret

```

subtraction:

```
write msg1, len ; Prompt for first number
read num, 17 ; Read first number

call ascii_hex ; Convert ASCII to hex
mov [num1], rbx ; Store number

write msg1, len ; Prompt for second number
read num, 17 ; Read second number

call ascii_hex ; Convert ASCII to hex
mov [num2], rbx ; Store number

mov rax, [num1]
mov rbx, [num2]

sub rax, rbx ; Perform subtraction
mov rbx, rax ; Store result

call display ; Display result
ret
```

multiplication:

```
write msg1, len ; Prompt for first number
read num, 17 ; Read first number

call ascii_hex ; Convert ASCII to hex
mov [num1], rbx ; Store number

write msg1, len ; Prompt for second number
read num, 17 ; Read second number

call ascii_hex ; Convert ASCII to hex
mov [num2], rbx ; Store number

mov rax, [num1]
mov rbx, [num2]

mul rbx ; Perform multiplication
mov rbx, rax ; Store result

call display ; Display result
ret
```

division:

```
write msg1, len ; Prompt for first number
read num, 17 ; Read first number

call ascii_hex ; Convert ASCII to hex
mov [num1], rbx ; Store number

write msg1, len ; Prompt for second number
```

read num, 17 ; Read second number

call ascii_hex ; Convert ASCII to hex
mov [num2], rbx ; Store number

mov rdx, 0 ; Clear remainder
mov rax, [num1]
mov rcx, [num2]

div rcx ; Perform division
mov rbx, rax ; Store quotient

call display ; Display result
ret

ascii_hex:

mov rbx, 0 ; Initialize result register
mov rsi, num ; Point to input string
mov rcx, 16 ; Process 16 characters

next:

rol rbx, 4 ; Shift left by 4 bits
mov al, [rsi] ; Load character
cmp al, '9' ; Check if numeric
jbe sub30h
sub al, 7h ; Convert A-F

sub30h:

sub al, 30h ; Convert 0-9
add bl, al ; Store converted value
inc rsi ; Move to next character
loop next ; Repeat
ret

display:

mov rsi, temp
mov rcx, 16

next1:

rol rbx, 4
mov al, bl
and al, 0Fh
cmp al, 9
jbe add30h
add al, 7h

add30h:

add al, 30h
mov [rsi], al
inc rsi
loop next1
write temp, 16
write endl, 1

ret